Claude Code prompt: MoE window-routing pilot

Nov 8, 2026 · @Archit Tamhane

Prompt part 1: Context, goal and ground rules

You are implementing a small, tightly controlled research pilot in this repository, a fork of nanoMoE (itself a fork of nanoGPT). Build the complete training harness: the model, training loop, configs, tests, cluster launchers and analysis script.

Research question. In a small mixture-of-experts language model, does routing the upper layers on a causal average of the last n tokens' hidden states give lower validation loss than standard per-token routing, at equal compute?

The two arms.

- **Baseline:** standard top-2 routing in every layer; each token's router sees only that token's hidden state.
- **Window arm:** identical, except in layers 5–8 (1-indexed) the router sees the causal mean of the hidden states at positions t−n+1 through t. The experts still process each token's own hidden state; only the routing decision changes.

Ground rules. These protect the comparison; don't relax them.

1. The only difference between arms is the router input in the window layers. Parameters, parameter creation order, initialization, data order per seed, balancing losses and all hyperparameters are identical. The single exception is that each arm uses the LR chosen by its own LR check.
2. No router noise, no expert capacity limits, no token dropping. If nanoMoE has any of these, remove them.
3. The comparison metric is pure next-token cross-entropy on validation data. Auxiliary and z-losses are logged separately and never added to a reported validation number.
4. Never weaken, skip or delete a test to make it pass. If a spec item seems wrong, or conflicts with the existing code in a way you can't resolve cleanly, stop and ask me.
5. Dependencies: only torch, numpy, tiktoken, pyarrow, matplotlib, pytest and the standard library. No wandb, no torch.compile, no external flash-attention packages.
6. Do not modify `data/slimpajama/prepare.py`. It has already produced the data.

7. Prefer simple, readable code. This is a pilot; correctness and clarity beat speed and generality.

Prompt part 2: Fixed facts

Data (already prepared, on the Engaging cluster) in /home/architta/orcd/scratch/moe-pilot **`/data/`:**

- `train.bin`: flat uint16 array of GPT-2 BPE token ids, exactly 600,000,000 tokens. Documents are concatenated with the end-of-text token 50256 after each one.
- `val_<source>.bin`: one uint16 file per source, same format. Sources: `arxiv`, `book`, `c4`, `commoncrawl`, `github`, `stackexchange`, `wikipedia`.
- `meta.json`: includes `train_tokens_total`, `train_share_by_source` (source → fraction of training tokens), `val_tokens_by_source`, `eot_token`. A copy may be at `data/slimpajama/meta.example.json` in this repo. Read source names and shares from `meta.json` at runtime; don't hard-code them beyond fallback defaults.
- The data was produced by `data/slimpajama/prepare.py` (read it for the exact format).

Where runs write output:

- Engaging: /home/architta/orcd/scratch/moe-pilot `/runs/<run_name>/`
- AWS fallback: `/home/ubuntu/runs/<run_name>/`, synced to `s3://none_yet/runs/<run_name>/`

Primary hardware: MIT Engaging (Slurm)

- Partition `mit_normal_gpu`: max job time 6 hours, default 2 hours (so always set it). Per-user limits: at most 2 GPUs, 32 CPUs and 515 GB of memory at once, and up to 64 submitted jobs.
- GPU gres labels: `b200` (about 180 GB each), `l40s` (48 GB), `h200`. Partition `mit_preemptable` allows 2-day jobs, but jobs can be preempted.
- Environment: `module load miniforge`, then `source "$(conda info --base)/etc/profile.d/conda.sh"` and `conda activate moe`. PyTorch built for CUDA 12.8 is installed there.
- **Plan A:** 2 B200 GPUs (2 Slurm jobs), each running 4 training processes concurrently on its one GPU, so all 8 main runs train at once.
- **Plan B:** L40S GPUs, 1 run per job (they queue, 2 at a time).

Fallback hardware: AWS us-east-2. g6e.12xlarge (4 × L40S) and g6e.xlarge (1 × L40S) instances on the AWS Deep Learning AMI with PyTorch. They have an IAM role with S3 access, and their shutdown behavior is set to terminate.

Laptop (where you are running): CPU only, with a Python venv in `.venv` containing torch, numpy, tiktoken, pyarrow, matplotlib and pytest. No GPU, no access to the cluster data. All tests must run here on CPU in under 2 minutes total, using small synthetic data they generate themselves.

Repository: https://github.com/archit10101/moe-window-routing, default branch `main`.

Prompt part 3: Model specification (`model.py`)

Keep nanoGPT's GPT skeleton and replace the MoE layer with exactly this.

Shape

Setting	Value
Layers / width / heads / context	8 / 512 / 8 / 1,024
Vocabulary	50,304 (nanoGPT's padded size; real ids are < 50,257)
Embeddings	Learned token + learned position embeddings, output head tied to the token embedding
Block	Pre-norm: x + attn(ln_1(x)), then x + moe(ln_2(x)); keep nanoGPT's LayerNorm without bias for ln_1, ln_2, ln_f
Attention	Causal, via `F.scaled_dot_product_attention(is_causal=True)`; no bias, no dropout
Every layer's MLP	Replaced by the MoE layer below (all 8 layers are MoE)
Dropout	0 everywhere

MoE layer (input h = ln_2(x), shape [B, T, C])

1. **Experts.** `SwiGLU(x) = W_down(silu(W_gate x) * (W_up x))`, no biases. One shared expert with hidden size 1,024 that every token goes through. Eight routed experts with hidden size 512 each.
2. **Router input.** Each layer has an integer `window`. If `window == 0`: r = h. If `window = n ⩾ 1`: r = the causal mean of h over positions max(0, t−n+1) .. t, computed on the [B, T, C] tensor before any flattening, in fp32:
   - S = cumsum(h.float(), dim=1)
   - S_prev = S shifted right by n positions along T, with zeros filled in at the start
   - count_t = min(t + 1, n)
   - r = ((S − S_prev) / count_t).to(h.dtype)
   - Put this in its own function, `causal_window_mean(h, n)`, so it can be tested directly. Windows never reach past the start of the sequence, and the batch dimension is never mixed.
3. **Router.** `router_norm = RMSNorm(C)` (learnable weight, initialized to ones, eps 1e-6), then `router = Linear(C, 8, bias=False)`, initialized normal with std 0.02. Both exist in every layer in both modes and are always applied: logits = router(router_norm(r)).
4. **Router math in fp32,** with autocast disabled for this block: logits = F.linear(input.float(), weight.float()); probs = softmax over all 8 experts; (top_p, top_i) = top-2 of probs; gates = top_p / top_p.sum(-1, keepdim=True).
5. **Dispatch.** Flatten h to [N, C] with N = B·T. For each expert e, gather the (token, slot) pairs where top_i == e, run the expert on those tokens, multiply by the matching gate and index_add into a zero output. No capacity, no dropping, no noise. Output = routed + shared(h), reshaped to [B, T, C].
6. **Balancing losses, per layer.**
   - counts = bincount(top_i.flatten(), minlength=8) as float.
   - Training mode: add counts to a non-persistent buffer `count_buf` and N·k to `slots_buf`; f = count_buf / slots_buf, so the load is pooled over all micro-batches of the current optimizer step. Eval mode: f = counts / (N·k), and the buffers are untouched.
   - P = probs.mean(0) over this micro-batch's tokens.
   - aux = 8 · Σ(f · P); z = mean(logsumexp(logits, -1)²).
   - Expose aux, z, f (detached) and counts for each layer after every forward, through nanoMoE's manager or a list on the model, your choice.
   - Provide `model.reset_load_buffers()`, called after every optimizer step.

7. **Routing statistics.** A model flag that, when enabled, accumulates per-layer expert counts for the analysis tables (used during final evaluation).

Initialization: nanoGPT's. Normal(0, 0.02) for Linear and Embedding weights; residual output projections (attention c_proj, every W_down) use std $0.02/\sqrt{2 \cdot n_layer}$; router std 0.02; norms at ones. Set `torch.manual_seed(seed)` before building the model. Module construction must not depend on `window`, so both arms build byte-identical initial weights for a given seed.

Parameter counts (print at startup, and test them):

Count	Expected
Non-embedding, total	71,348,736
Non-embedding, active per token (shared + 2 of 8 routed)	33,599,488
All parameters (tied head)	97,628,672

If your counts differ by more than 1%, explain why before continuing.

Prompt part 4: Training loop specification (`train.py`)

Entry point: `python train.py config/runs/<name>.py [--key=value ...]`, using the existing `configurator.py`. One process, one device (a CUDA GPU, or CPU for tests). Distributed training isn't needed; remove the DDP paths or leave them unused, but nothing may depend on them.

Config keys and defaults (in `config/pilot_base.py`):

Key	Default	Notes
run_name	(required)	Folder name under out_root
out_root	/home/architta/orcd/scratch/moe-pilot`/runs`	
data_dir	/home/architta/orcd/scratch/moe-pilot`/data`	

Key	Default	Notes
seed	0	Controls init and data order
window_layers	[0]*8	Per-layer router window; window arm: [0,0,0,0,n,n,n,n]
lr	(required)	Peak learning rate
min_lr_ratio, warmup_frac	0.1, 0.02	Linear warmup, then cosine decay to lr × 0.1 at the last step
weight_decay, beta1, beta2, grad_clip	0.1, 0.9, 0.95, 1.0	AdamW (fused on CUDA); decay on params with dim ⩾ 2 only
batch_size, micro_batch_size, block_size	64, 64, 1024	grad_accum = batch_size / micro_batch_size
total_tokens	(required)	max_steps = ceil(total_tokens / (batch_size · block_size)); 500M → 7,630 steps, 100M → 1,526
aux_coef, z_coef	0.01, 0.001	
log_interval, eval_interval	50, 500	Steps
eval_batches, final_eval_batches, eval_batch_size	20, 100, 16	Per source
ckpt_interval_minutes	15	
dtype	bfloat16	Autocast on CUDA; plain fp32 on CPU
num_threads	4	`torch.set_num_threads`

Key	Default	Notes
benchmark_steps	0	If > 0: benchmark mode, see below
resume	True	

Data sampling. Memory-map `train.bin`. For optimizer step s, use `rng = np.random.default_rng([seed, s])` to draw batch_size offsets in [0, len − block_size − 1); x = data[o : o + T], y = data[o + 1 : o + T + 1] as int64. Split into micro-batches in order. This makes data order a pure function of (seed, step), so resuming is exact. On CUDA use pinned memory and non-blocking copies.

Validation.

- Read the sources and `train_share_by_source` from `meta.json`. For each source, draw final_eval_batches × eval_batch_size offsets once, with `np.random.default_rng([12345, i])` where i is the source's index in sorted order.
- Periodic evals use the first eval_batches × eval_batch_size of those offsets; the final eval uses all of them. Skip, with a warning, any source whose file is shorter than block_size + 1.
- Per-source cross-entropy is the fp32 mean over all target tokens. `val_overall` = Σ share_s · CE_s / Σ share_s over the sources present.
- Evaluate in eval mode under no_grad (the load buffers must not change). Evaluate at step 0, every eval_interval steps, and at the end.

Loss and step. Per micro-batch: loss = CE + aux_coef · Σ_layers aux + z_coef · Σ_layers z, divided by grad_accum, then backward. After grad_accum micro-batches: clip to grad_clip, optimizer step, `model.reset_load_buffers()`, `zero_grad(set_to_none=True)`. Call `.item()` only on logging steps.

Log file: `<run_dir>/log.jsonl`, one JSON object per line, flushed after every write.

- `header`: run_name, the full config, git commit hash and dirty flag, hostname, GPU name, torch and CUDA versions, parameter counts, max_steps, tokens_per_step, start time.
- `train` (every log_interval): step, tokens, ce, aux, z (each averaged over the step's micro-batches), lr, grad_norm, tok_per_s, load_max and load_min (lists of 8, one per layer), time.
- `eval`: step, tokens, val (source → CE), val_overall.

- `resume`: step. Written whenever a run resumes.
- `final`: step, tokens, val, val_overall, wall_time_s.

End of run. Write the final checkpoint, `routing_counts.json` (per layer: source → 8 expert counts on the final eval batches, plus window_layers), `final.json`, then an empty `done` file. If `done` already exists at startup, print a message and exit 0, so relaunching is always safe.

Checkpoints. Every ckpt_interval_minutes and at the end, save the model, optimizer, step, config and RNG states (python, numpy, torch, cuda) to `ckpt/latest.pt.tmp`, then `os.replace` it to `ckpt/latest.pt`. Keep only the latest. With resume=True and a checkpoint present, load it and continue from the next step.

Signals. On SIGUSR1 or SIGTERM, set a flag. At the end of the current optimizer step, save a checkpoint and exit with code 99, meaning interrupted but resumable.

Benchmark mode (benchmark_steps > 0): run that many training steps without eval or checkpoints and without writing a run folder. Print tokens per second averaged after the first 5 steps, and peak GPU memory in GB (`torch.cuda.max_memory_allocated`). Then exit.

Prompt part 5: Configs and run matrix

- `config/pilot_base.py`: every default from part 4. `micro_batch_size` lives only here, so the memory-check result is changed in one place for every run.
- `config/chosen_lr.py`: `LR_BASE = None` and `LR_WIN = None`. Main-run configs read their LR from here and stop with a clear message if it is still None.
- `config/runs/<name>.py`: one small file per run that loads `pilot_base.py` and overrides only what differs. Generate them with `scripts/make_configs.py` or write them by hand, but they must match this table exactly.

Run names	window_layers	seed	lr	total_tokens
lr-base-1e-3, lr-base-2e-3, lr-base-4e-3	[0]*8	0	1e-3, 2e-3, 4e-3	100M
lr-win16-1e-3, lr-win16-2e-3, lr-win16-4e-3	[0,0,0,0,16,16,16,16]	0	1e-3, 2e-3, 4e-3	100M

Run names	window_layers	seed	lr	total_tokens
base-s0, base-s1, base-s2	[0]*8	0, 1, 2	LR_BASE	500M
win16-s0, win16-s1, win16-s2	[0,0,0,0,16,16,16,16]	0, 1, 2	LR_WIN	500M
win1-s0 (sanity check)	[0,0,0,0,1,1,1,1]	0	LR_WIN	500M
win64-s0 (window sweep)	[0,0,0,0,64,64,64,64]	0	LR_WIN	500M
base-s3, base-s4, win16-s3, win16-s4 (optional extra seeds)	as their arm	3, 4	as their arm	500M
smoke-gpu-base, smoke-gpu-win16	as their arm	0	2e-3	200 steps × 65,536 tokens
smoke-cpu	tiny model (see part 6), window [0, 4]	0	3e-3	small; finishes in under a minute on CPU

`smoke-cpu` uses a synthetic data folder made by `scripts/make_synthetic_data.py --out <dir>`: random uint16 tokens below 1,000, a small `train.bin`, three `val_<source>.bin` files and a matching `meta.json`. The tests use the same generator.

Prompt part 6: Tests (`tests/`, pytest, CPU only)

Tiny test config: 2 layers, width 64, 4 heads, context 128, 4 routed experts (hidden 64), shared expert hidden 128, top-2. The window model uses window_layers [0, 4]. Data comes from the synthetic generator in part 5, written to a pytest temp folder. The whole suite must pass in under 2 minutes.

Test	What must hold
`test_window_mean_matches_naive`	`causal_window_mean` equals a plain Python loop over positions (allclose, 1e-5) for n = 1, 3, 128, 200, on a batch of 2 with different sequences

Test	What must hold
`test_no_future_leakage`	Window model, one 128-token sequence: change the token at position 100. Router logits at positions 0–99 in every layer are exactly equal (torch.equal, fp32), and some logits at positions ⩾ 100 differ
`test_no_batch_leakage`	Changing sequence 2 of a batch doesn't change any router logit of sequence 1
`test_identical_init_across_modes`	For the same seed, window_layers [0,0] and [0,4] give identical state_dict keys, shapes and values
`test_n1_equals_per_token`	Same seed and data: baseline vs window_layers [0,1] for 20 training steps; losses match within 1e-4 at every step
`test_router_learns_from_lm_loss`	With aux_coef = z_coef = 0, one backward pass on cross-entropy gives a non-zero gradient on every router weight
`test_pooled_load_counts`	With micro_batch_size = batch_size / 4: after 4 training-mode forwards, f equals the pooled counts / pooled slots; `reset_load_buffers()` zeroes them; an eval-mode forward doesn't change them
`test_overfit_single_batch`	300 steps on one fixed batch (4 × 32 tokens): final CE < 10% of initial CE
`test_balance_holds`	200 steps on random data: every expert's load fraction in every layer is between 0.02 and 0.5
`test_resume_is_exact`	20 straight steps vs 10 steps + checkpoint + fresh process state + resume + 10 steps: final losses equal (allclose, 1e-6)
`test_sampler_deterministic`	Same (seed, step) gives the same offsets; a different seed gives different offsets
`test_param_counts_real_config`	Building the real 8-layer config (no data needed) gives the part 3 counts within 1%
`test_end_to_end_cpu`	`train.py` on the smoke-cpu config writes log.jsonl with header, train, eval and final lines, plus routing_counts.json, final.json and done; a relaunch exits immediately because of done

Test	What must hold
`test_analysis_runs`	`analysis/analyze.py` on a folder of fake runs (copies of the smoke-cpu output relabeled as base-s0..s2, win16-s0..s2, win1-s0, win64-s0) produces every output file and a verdict

Prompt part 7: Launchers

All shell scripts use bash with `set -uo pipefail`, take arguments rather than editing the script, and pass `bash -n`.

Engaging, Slurm (primary): `scripts/slurm/`

- **`run_group.sbatch`** runs a group of k runs on one GPU in one job.
  - The run names are passed as arguments.
  - It loads the conda environment as in part 2 and sets `OMP_NUM_THREADS=4`.
  - For each run without a `done` file, it starts `python train.py config/runs/<name>.py` in the background, appending stdout and stderr to `<run_dir>/stdout.log`. All k processes share the job's single GPU.
  - It traps SIGUSR1 and SIGTERM and forwards SIGUSR1 to the children, which checkpoint and exit with 99. Then it waits for all children.
  - Afterwards, if any run in the group lacks `done` and this job has been requeued fewer than 3 times (a counter file in `<out_root>/slurm/`), it calls `scontrol requeue $SLURM_JOB_ID`.
  - Sbatch options: `--requeue` and `--signal=B:USR1@300`. Slurm output goes to `<out_root>/slurm/%x-%j.out`.
- **`submit.sh <group-name> <gpu-type> <time> <run> [run ...]`** calls sbatch on `run_group.sbatch` with `-p mit_normal_gpu --gres=gpu:<gpu-type>:1 -c <4·k, at most 16> --mem=<min(48·k, 200)>G -t <time> -J <group-name>`. It refuses to submit if a run's config file is missing, or if a main run's LR is still None.
- **Convenience scripts** that call `submit.sh`:
  - `submit_lr_check.sh <gpu-type>`: two jobs of 3 runs each on B200 (the base LRs, then the window LRs); with `140s`, six 1-run jobs.

  - `submit_main.sh <gpu-type>`: on B200, two jobs of 4: job A = base-s0, win16-s0, base-s1, win16-s1; job B = base-s2, win16-s2, win1-s0, win64-s0. With `l40s`, eight 1-run jobs. Default time 05:30:00.
  - `submit_extra_seeds.sh <gpu-type>`: base-s3, win16-s3, base-s4, win16-s4.
  - `submit_smoke.sh <gpu-type>`: smoke-gpu-base and smoke-gpu-win16 in one job; time 00:30:00.
- **`benchmark.sh <gpu-type>`**: one job that runs smoke-gpu-base with benchmark_steps=30, first alone, then 4 copies at once on the same GPU, and prints tokens per second and peak memory for each case. This tells me whether 4 runs per B200 is efficient, and confirms the memory check.
- **`status.sh`**: for every run folder, prints the latest step, tokens, train CE, tok/s, latest val_overall and whether it's done, then shows `squeue -u $USER`.

The per-user limits allow 2 jobs at once (32 CPUs, 2 GPUs), so each B200 job may use at most 16 CPUs.

AWS (fallback): `scripts/aws/`

- **`setup_instance.sh`**: activates the Deep Learning AMI's PyTorch environment (detect it; print clear instructions if it isn't found), installs tiktoken, pyarrow and matplotlib, clones or pulls the repo into `/home/ubuntu/code`, syncs `s3://none_yet/data/` to `/home/ubuntu/data`, and prints `nvidia-smi`.
- **`launch_run.sh <run_name> <gpu_index> [--no-shutdown]`**:
  - Runs training with `CUDA_VISIBLE_DEVICES=<gpu_index>`, `data_dir=/home/ubuntu/data` and `out_root=/home/ubuntu/runs`.
  - On a non-zero exit, retries with resume up to 3 times.
  - In the background, syncs the run folder to S3 every 5 minutes, excluding `ckpt/`, and includes the checkpoint every 15 minutes.
  - On completion: a final full sync, then a check that `done` exists in S3.
  - Unless `--no-shutdown` is given, runs `sudo shutdown -h now` only when every run listed in `/home/ubuntu/active_runs.txt` has a `done` file.
- **`launch_instance.sh <run> [run ...]`**: writes `active_runs.txt`, then starts `launch_run.sh` for each run on GPUs 0..k−1 with nohup, logging to `/home/ubuntu/launch_logs/`.

Prompt part 8: Analysis script (`analysis/analyze.py`)

Usage: `python analysis/analyze.py --runs_dir <dir> --out_dir <dir> [--lr_check [--write]]`. Only numpy, matplotlib, json and csv. Read each run's `log.jsonl`; after a resume, keep the last record for each step.

LR-check mode (`--lr_check`). For each arm, list the lr-* runs' final val_overall and pick the LR with the lowest value. If another LR is within 0.01 of the best, pick the lowest such LR (safer for a 5× longer run). Print the result. With `--write`, also write `config/chosen_lr.py`.

Main mode outputs:

1. `sanity.png`: val_overall vs tokens for base-s0..s2 (thin lines) and win1-s0 (bold).
2. `main_comparison.png`: val_overall vs tokens for base and win16, each drawn as the mean across seeds with a min–max band.
3. `per_source.csv` and `per_source.md`: final CE per source for every run, plus base mean, win16 mean and win16 − base.
4. `window_sweep.png`: final val_overall at windows 1, 16 (mean across seeds) and 64, with the baseline mean as a horizontal line and a ±2s band (s defined below).
5. `loads.png`: per-layer max and min expert load over training, for base-s0 and win16-s0.
6. `routing_heatmap.png`: for layer 7, source × expert routing shares (each source's row normalized to 1), base-s0 and win16-s0 side by side. Also `routing_mi.csv`: mutual information between source and expert, per layer, per run.
7. `summary.md`: every number below and the verdict.

Decision rule (exactly this; use every available seed, at least 2 per arm):

- Final values: B = the baseline seeds' final val_overall, W = the win16 seeds' final val_overall.
- s = √((var(B) + var(W)) / 2), with sample variance (ddof = 1).
- gap = mean(B) − mean(W). Positive means the window arm is better.
- **Sanity:** |win1-s0 final − mean(B)| ⩽ 2s. If this fails, the verdict is "INVALID: sanity check failed", whatever else holds.
- **PROMISING:** gap > 2s, and win64-s0 final ⩽ mean(B) + 2s.
- **NEGATIVE:** gap < −2s.
- **INCONCLUSIVE:** anything else.

State the verdict, gap, s and every final value in `summary.md`, and also print them.

Prompt part 9: Deliverables, working style and definition of done

Deliverables

- `model.py`, `train.py`: changed as specified.
- `config/pilot_base.py`, `config/chosen_lr.py`, `config/runs/*.py` (and `scripts/make_configs.py` if you generate them).
- `scripts/make_synthetic_data.py`.
- `tests/` with every test in part 6.
- `scripts/slurm/` and `scripts/aws/` as in part 7.
- `analysis/analyze.py`.
- `README_PILOT.md`: the exact commands, in order, for everything after you finish:
  1. Running the tests.
  2. On Engaging: cloning the repo into /home/architta/orcd/scratch/moe-pilot`/code`, activating the environment, the GPU smoke test, the benchmark and memory check (and how to switch `micro_batch_size` to 16 if peak memory is too high), the LR check, choosing LRs with `analyze.py --lr_check --write`, submitting the main runs, monitoring with `status.sh`, the optional extra seeds, and running the analysis.
  3. The AWS fallback equivalents.
  4. A short troubleshooting list.

Working style

1. Read the existing code first, then propose a plan and wait for my approval.
2. Build in this order, running the relevant tests after each stage: model → model tests → training loop → training tests → configs → launchers → analysis → README.
3. Commit after each stage with a clear message. Don't push; I will.
4. Don't run anything that needs a GPU, the cluster or the network. You can't reach them from here.
5. If something in this spec is ambiguous, or seems wrong, ask instead of guessing. List any deviations you made, and why, in your final summary.

Definition of done

- [ ] `pytest -q` passes on this CPU laptop in under 2 minutes, with no skipped tests.
- [ ] `python scripts/make_synthetic_data.py --out /tmp/synth` followed by training on the smoke-cpu config completes and writes every output file.

- [ ] `bash -n` passes for every shell script.
- [ ] Parameter counts printed for the real config match part 3 within 1%.
- [ ] `README_PILOT.md` is complete enough that I can go from a fresh Engaging login to submitted main runs by following it line by line.
- [ ] A final summary listing what you built, the test results, any deviations from this spec, and anything I need to decide.